



VITyarthi

EVALUATED COURSE PROJECT

BANKING PROGRAM

Student Name	HARSHIT PARMAR
Registration Number	26MIP10070
Course / Subject	Introduction to problem solving and programming
Faculty Name	Devendra Kumar Vedi

Prepared as a course project report based on the working program developed in Python.S

1. Introduction

The Banking Program is a small command-line Python application that demonstrates how basic programming concepts can be combined into a simple real-world style workflow. The program starts with a balance of zero and gives the user four choices: show the balance, deposit money, withdraw money, or exit.

The implementation is function based. `show_balance()` displays the current amount, `deposit()` checks a deposit value and returns it, `withdraw()` checks the amount against the balance and returns it when valid, and `main()` controls the menu loop. This structure keeps the code easy to follow for a beginner.

1.1 Project Overview

The program uses standard terminal input and output. No graphical interface, external database, or third-party package is required. A user selects an option, enters an amount when necessary, receives the result, and returns to the menu until exit is chosen.

1.2 Course Relevance

The project uses functions, parameters, return values, variables, if/elif/else statements, a while loop, formatted strings, keyboard input, and basic validation. These are core Python programming concepts and are visible directly in the source code.

1.3 Executive Summary

The project is intentionally compact. It is not intended to model a commercial banking platform. Its purpose is to demonstrate a complete input-process-output cycle in a small, understandable program.

2. Problem Statement, Objectives and Scope

2.1 Problem Statement

A beginner needs a programming example where separate concepts work together in one useful flow. Banking is a suitable example because it has simple rules that can be represented with conditions: a balance can increase through deposits, decrease through withdrawals, and withdrawals should not exceed available funds.

The project therefore implements a menu-driven Banking Program that accepts a user choice, performs the selected balance operation, and repeats the interaction until the exit option is selected.

2.2 Objectives

- Build a working command-line banking example in Python.
- Use functions to separate display, deposit, withdrawal, and control logic.
- Maintain a current balance during one program session.
- Validate negative amounts and insufficient funds.
- Handle invalid menu choices without ending the program.
- Keep the application simple enough for classroom demonstration.

2.3 Scope

The current scope covers one balance, deposits, withdrawals, balance display, menu navigation, and exit. There are no customer accounts, authentication, transaction history, database storage, or multi-user features in the supplied implementation.

2.4 Target User

The target user is a student or beginner learning Python and basic application logic. The program can also be demonstrated in a lab or viva to show how functions and loops work together.

3. Functional Requirements

The following requirements are directly supported by the supplied program.

ID	Function	Expected behavior
FR-01	Show Balance	Displays the current balance with two decimal places.
FR-02	Deposit	Accepts a deposit amount and adds it when the value is valid.
FR-03	Withdraw	Subtracts a withdrawal when enough funds are available.
FR-04	Insufficient Funds	Rejects a withdrawal larger than the current balance.
FR-05	Amount Validation	Rejects negative deposit and withdrawal values.
FR-06	Menu Loop	Repeats the menu while the program is running.
FR-07	Exit	Stops the loop when choice 4 is selected.
FR-08	Invalid Menu Choice	Displays a message for a choice outside 1-4.

3.1 Typical User Flow

1. Start the program and view the menu.

2. Choose Deposit and enter a positive amount.
3. Choose Show Balance to verify the new balance.
4. Choose Withdraw and enter an amount not greater than the balance.
5. View the remaining balance.
6. Choose Exit.

3.2 Functional Boundary

The balance exists only while the program is running. When the application is closed, the balance is reset because no persistent storage is implemented.

4. Non-Functional Requirements and Technical Design

Requirement	Implementation in current program
Usability	Simple four-option text menu and clear prompts.
Performance	Immediate response because only small in-memory values are processed.
Reliability	Invalid values return zero and do not change the balance.
Maintainability	Separate functions isolate each major operation.
Portability	Uses standard Python language features and terminal I/O.
Simplicity	No third-party packages or external services.

4.1 Technology Used

Technology	Purpose
Python 3	Complete application logic.
Terminal / Command Prompt	User interface.
Standard input/output	Read choices and amounts; display results.

4.2 Program Structure

The program contains four major functions: `show_balance()`, `deposit()`, `withdraw()`, and `main()`. `main()` owns the running state; the helper functions perform focused tasks and return control to the main loop.

4.3 Runtime Data

The key variable is balance. It starts at 0 and is updated after a successful deposit or withdrawal. Other variables include choice, amount, and is_running.

5. System Architecture and Workflow

System Architecture

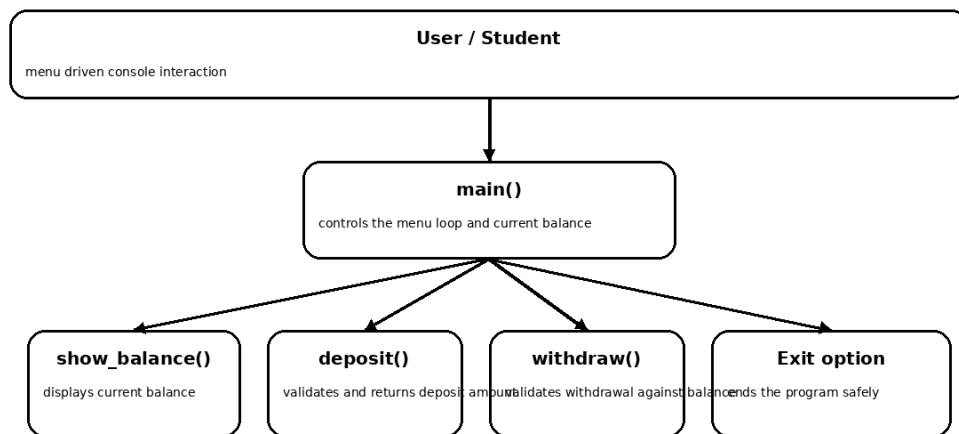


Figure 1. High-level architecture of the Banking Program.

The user interacts with **main()**. The selected option routes execution to a focused function or to the exit branch. The helper function returns control to **main()**, which then shows the menu again.

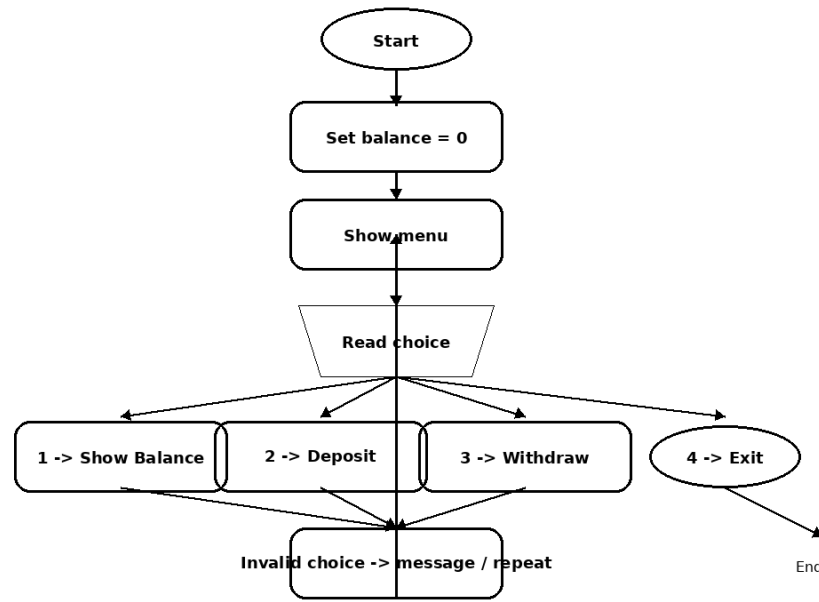
Process Workflow

Figure 2. Workflow from start to exit.

6. UML-Oriented Design Views

Use Case Diagram

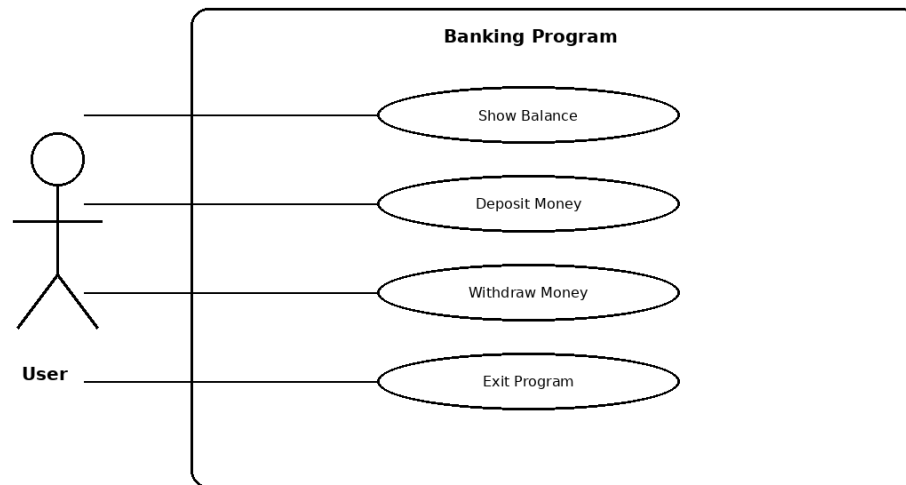


Figure 3. Use case view of the application.

The single actor is the user. The four main use cases match the four menu choices: show balance, deposit, withdraw, and exit.

Sequence Diagram - Deposit Example

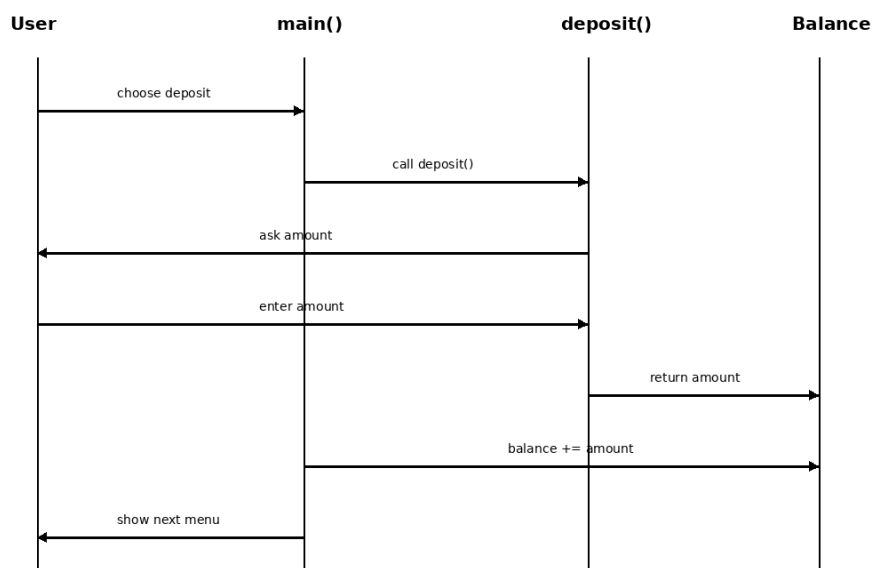


Figure 4. Example sequence for a deposit operation.

For a deposit, the user selects the option, `main()` calls `deposit()`, the amount is entered and validated, the amount is returned, and `main()` updates balance.

7. Algorithm and Implementation Details

7.1 Main Algorithm

7. Set balance to 0 and is_running to True.
8. Display the menu.
9. Read the choice.
10. Choice 1 -> show_balance(balance).
11. Choice 2 -> balance += deposit().
12. Choice 3 -> balance -= withdraw(balance).
13. Choice 4 -> set is_running to False.
14. Any other choice -> print an invalid-choice message.
15. Repeat until is_running becomes False.

7.2 Deposit Logic

```
def deposit():  
    amount = float(input("Enter an amount to be  
deposited: "))  
    if amount < 0:  
        print("That's not a valid amount")  
        return 0  
    else:  
        return amount
```

The function returns either the valid amount or zero. Returning zero prevents an invalid deposit from changing the balance.

7.3 Withdrawal Logic

```
def withdraw(balance):  
    amount = float(input("Enter amount to be withdrawn:  
"))
```

```
if amount > balance:
    print("Insufficient funds")
    return 0
elif amount < 0:
    print("Amount must be greater than 0")
    return 0
else:
    return amount
```

The withdrawal function checks available funds and the sign of the entered amount before returning a value.

8. Source Code Walkthrough

8.1 Balance Display

```
def show_balance(balance):  
    print(f"Your balance is ${balance:.2f}")
```

The `:.2f` format specifier keeps the displayed amount at two decimal places.

8.2 Main Loop

```
def main():  
    balance = 0  
    is_running = True  
  
    while is_running:  
        ...
```

`main()` stores the current balance and controls the repeated menu. The balance changes only through the values returned by `deposit()` and `withdraw()`.

8.3 Program Entry Point

```
if __name__ == '__main__':  
    main()
```

The entry-point condition calls `main()` when the file is executed directly. The corrected form fixes the original draft error where `_main_` was written instead of `__main__`.

8.4 Design Choice

A function-based design is suitable because the project is intentionally small. Using separate functions makes each task easier to understand and keeps the menu logic inside `main()`.

9. Testing Approach and Results

Testing focuses on the visible menu and the balance rules present in the source code.

ID	Scenario	Input	Expected result
TC-01	Start	Run file	Menu appears
TC-02	Show balance	1 at start	Shows \$0.00
TC-03	Valid deposit	2, then 500	Balance becomes \$500.00
TC-04	Negative deposit	2, then -50	Invalid amount; balance unchanged
TC-05	Valid withdrawal	3, then 200 from 500	Balance becomes \$300.00
TC-06	Overdraw	3, then 600 from 500	Insufficient funds; balance unchanged
TC-07	Negative withdrawal	3, then -20	Invalid amount; balance unchanged
TC-08	Bad menu choice	9	Invalid choice; menu repeats
TC-09	Exit	4	Thank-you message and program ends

9.1 Sample Console Session

```
Banking Program
1.Show Balance
2.Deposit
3.Withdraw
```

4.Exit

```
Enter your choice (1-4): 2
Enter an amount to be deposited: 500
Enter your choice (1-4): 1
Your balance is $500.00
Enter your choice (1-4): 3
Enter amount to be withdrawn: 200
Enter your choice (1-4): 1
Your balance is $300.00
```

9.2 Testing Note

The core cases pass when the corrected `__main__` entry point is used. The current source still assumes that values entered for amount can be converted to float; non-numeric text is a possible future input-handling improvement.

10. Challenges, Learnings and Limitations

10.1 Challenges Faced

- Keeping the balance correct after each operation.
- Preventing invalid amounts from changing the balance.
- Checking sufficient funds before a withdrawal.
- Keeping the menu active until Exit is selected.
- Correcting the program entry-point statement.

10.2 Learnings

- Functions divide a program into understandable tasks.
- Return values can carry validated results back to main().
- A while loop is useful for a repeated menu.
- Conditions can model simple transaction rules.
- Input validation matters because user input affects the balance.

10.3 Limitations

- One balance per execution.
- No customer accounts or authentication.
- No persistent storage.
- No transaction history.
- No transfer feature or GUI.
- Non-numeric amount input may raise a ValueError.

10.4 Scope of the Current Version

The project is a foundational programming exercise rather than a production banking system. Its main value is showing a complete and testable input-process-output flow with basic Python logic.

11. Future Enhancements, Conclusion and References

11.1 Future Enhancements

- Add try/except for non-numeric amount input.
- Add customer accounts and PIN authentication.
- Store balances in a database.
- Add transaction history and timestamps.
- Support account-to-account transfers.
- Add admin/reporting functions.
- Add a graphical or web interface.

11.2 Conclusion

The Banking Program successfully demonstrates a small banking workflow using standard Python. It accepts user choices, maintains a balance, performs deposits and withdrawals, rejects withdrawals above the available balance, and exits cleanly. Its function-based structure makes the program easy to understand and extend.

The current version can be treated as a base for a larger project. The natural next step is better input handling followed by persistent account data, while keeping the simple menu-driven approach.

11.3 References

- Python 3 documentation for `input()`, `print()`, `float()`, functions, loops, and conditional statements.
- Course material on Python functions, loops, conditions, and basic input/output.
- The project source code used for implementation and testing.

11.4 Submission Note

README.md should be kept at the repository root. The report and source code should describe the same implemented behavior so that the repository, demonstration, and documentation remain consistent.